

Assignment 1 – Blinking LED med ATmega32

Oskar Hellebust-Nygaard

12. desember 2025

Innhold

1	Mål	2
2	Teori	2
3	Utstyr og oppkobling	2
4	Kode	3
5	Gjennomføring	4
6	Resultat	4
7	Diskusjon	4
8	Konklusjon	4

1 Mål

Målet med denne oppgaven var å skrive et enkelt C-program for å få to LEDs til å blinke annenhver gang med en ATmega32. Oppgaven gav en introduksjon til hvordan man setter opp digitale utganger i C og hvordan man kompilerer og laster opp programmet til mikrokontrolleren ved hjelp av en USBtiny-programmerer.

2 Teori

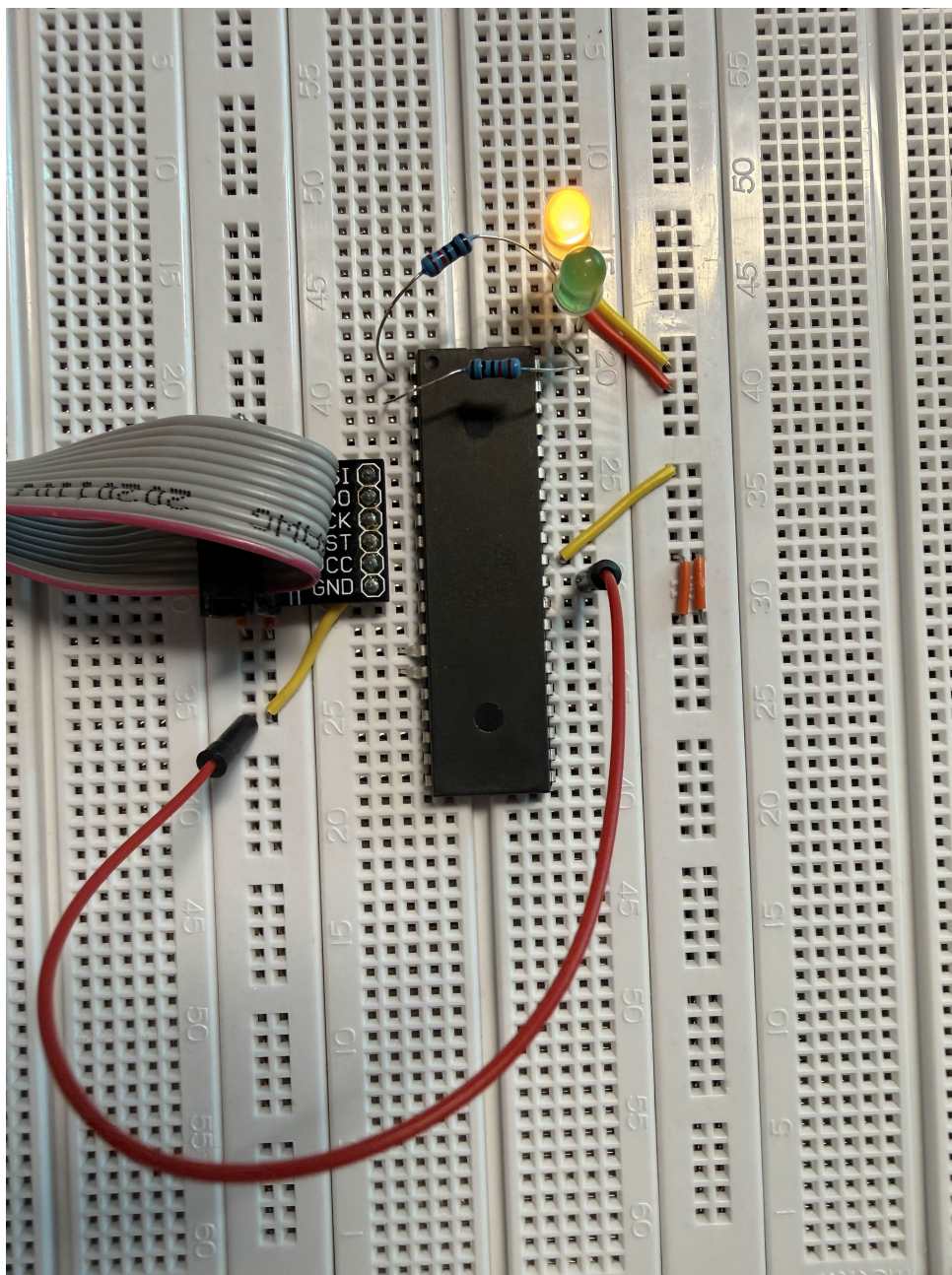
En LED lyser når strøm går fra anoden til katoden. For å beskytte dioden brukes en seriemotstand, i dette tilfellet på omtrent $220\ \Omega$. Mikrokontrolleren styrer LED-ene ved å sette utgangspinnene på PORTB til høy eller lav logisk verdi. Programmet er skrevet i C og kompiles til en `.hex`-fil som lastes opp via `avrdude`.

3 Utstyr og oppkobling

- ATmega32 mikrokontroller (DIP-40)
- USBtinyISP Pocket Programmer
- Breadboard og ledninger
- Én grønn og én gul LED
- To $220\ \Omega$ motstander

Oppkoblingen ble gjort slik:

- PB0 (pin 1) $\rightarrow$ motstand $\rightarrow$ LED1 $\rightarrow$ GND
- PB1 (pin 2) $\rightarrow$ motstand $\rightarrow$ LED2 $\rightarrow$ GND
- VCC til pin 10 og 30, GND til pin 11 og 31
- ISP-tilkobling mellom USBtiny og ATmega32 (MOSI, MISO, SCK, RST)



Figur 1: Oppkobling av ATmega32 med to LED-er (PB0 og PB1).

4 Kode

Koden under får LED-ene til å blinke annenhver gang med en delay på omtrent 500 ms.

```
#define F_CPU 1000000UL // 1 MHz for _delay
#include <avr/io.h>
#include <util/delay.h>

int main() {
    // Sett PORTB0 og PORTB1 som utganger
    DDRB |= (1 << PB0) | (1 << PB1);
```

```

// Skru av begge LED for starttilstand
PORTB &= ~((1 << PB0) | (1 << PB1));

while (1) {

    // Skru på PB0, av PB1
    PORTB |= (1 << PB0);    // PB0 HIGH
    PORTB &= ~(1 << PB1);    // PB1 LOW
    _delay_ms(500);

    // Skru på PB1, av PB0
    PORTB |= (1 << PB1);    // PB1 HIGH
    PORTB &= ~(1 << PB0);    // PB0 LOW
    _delay_ms(500);
}
}

```

5 Gjennomføring

Koden ble skrevet og kompilert i Microchip Studio. USBtiny-programmereren ble brukt til å laste opp `.hex`-filen til ATmega32. Etter vellykket kommunikasjon begynte LEDene å blinke rett etter programmering.

6 Resultat

Begge LED-ene blinket vekselvis med omtrent et halvt sekunds intervall, som forventet. Oppkoblingen var stabil, og strømforsyningen fra programmereren (5V) var tilstrekkelig.

7 Diskusjon

Det viktigste læringspunktet var forståelsen av hvordan DDR-registeret konfigurerer porter som inngang eller utgang, samt hvordan PORT-registeret brukes for å endre logiske nivåer. Et vanlig feilpunkt underveis var at AVCC (pin 30) ikke var koblet til VCC, noe som førte til at mikrokontrolleren ikke reagerte. Etter korreksjon av dette og riktig valg av strøm på USBtiny-programmereren fungerte alt som planlagt.

8 Konklusjon

Oppgaven ble gjort som planlagt. Mikrokontrolleren ble programmert med et enkelt blinkeprogram i C, og koblingen fungerte stabilt. Det ga en grunnleggende forståelse for bruk av digitale utganger, kompilering og programmering av ATmega-mikrokontrollere med avrdude.